

Fundamentos de Comunicaciones Digitales de Alta Velocidad

Trabajo Práctico N° 1

Ejercicio 1. Un filtro ampliamente utilizado en comunicaciones digitales es el *coseno realzado* (*Raised Cosine*, RC).

1. Genere una función en **Python** que reciba como parámetros:

- el *symbol rate* (equivalente a $1/T$),
- la frecuencia de muestreo del filtro,
- el *rolloff* o exceso de ancho de banda β ,
- la cantidad de *taps* que se desean calcular.

La función deberá devolver la respuesta al impulso del filtro RC.

2. Genere una figura donde se superponga el *plot* de varias respuestas al impulso para:

- BR: 32GBd
- fs: 4*BR
- β variable

¿Qué tienen en común todas las respuestas? ¿En qué se diferencian unas de otras?

3. Usando la función `numpy.fft.fft()`, grafique la magnitud de la respuesta en frecuencia de todas las respuestas al impulso calculadas en el punto anterior.

¿Qué tienen en común todas las respuestas? ¿En qué se diferencian unas de otras?

Ejercicio 2. Otro filtro ampliamente utilizado en comunicaciones digitales es el *raiz coseno-realzado* (*Root-Raised Cosine*, RRC).

1. Genere una función en **Python** que reciba como parámetros:

- el *symbol rate* (equivalente a $1/T$),
- la frecuencia de muestreo del filtro,
- el *rolloff* o exceso de ancho de banda β ,
- la cantidad de *taps* que se desean calcular.

La función deberá devolver la respuesta al impulso del filtro RRC.

2. Genere una figura donde se superponga el *plot* de varias respuestas al impulso para:

- BR: 32GBd

- $fs: 4*BR$
- β variable

¿Qué tienen en común todas las respuestas? ¿En qué se diferencian unas de otras?
Comparar con el RC.

3. Usando la función `numpy.fft.fft()`, grafique la magnitud de la respuesta en frecuencia de todas las respuestas al impulso calculadas en el punto anterior.

¿Qué tienen en común todas las respuestas? ¿En qué se diferencian unas de otras?
Comparar con el RC.

4. Convolucione dos filtros RRC y compruebe que el resultado es un RC.

Ejercicio 3. Una forma muy útil de modelar un transmisor elemental PAM- M se muestra en la Fig. 2. El generador de bits aleatorios entrega una cadena de bits que será mapeada a una secuencia de símbolos. Estos símbolos son transmitidos a una velocidad llamada *baud-rate* (BR) o *symbol-rate*, siendo T el período entre símbolos ($T = 1/BR$).

El filtro RC actúa como filtro transmisor, cuya finalidad es la conformación espectral (*pulse shaping*).

Para modelar este filtro en **Python**, es necesario realizar el filtrado en dos pasos:

- Sobremuestrear la señal a una tasa N (insertar $N - 1$ ceros entre muestras de la señal original).
- Filtrar la señal con el filtro elegido, muestreado en los instantes $n(T/N)$.

Nótese que la frecuencia de muestreo del filtro es N veces el *baud-rate*.

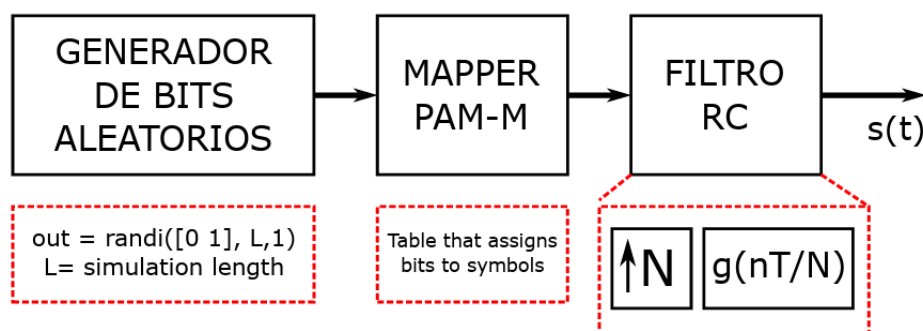


Figure 1: Modelo de transmisor PAM- M .

Implemente en **Python** un script que modele el sistema propuesto y resuelva los siguientes puntos:

1. Alinee la salida del filtro con los símbolos transmitidos (tenga en cuenta que se encuentran a distinta tasa de muestreo) y grafique ambas señales superpuestas utilizando:

- `matplotlib.pyplot.plot()` para la señal $s(t)$,

- `matplotlib.pyplot.stem()` para los símbolos transmitidos.

¿Se pueden detectar los símbolos transmitidos a partir de la señal $s(t)$? ¿Hay ISI?

2. Repita el punto anterior utilizando un filtro *Root Raised Cosine* (RRC) en lugar de un filtro RC. ¿Se pueden detectar los símbolos transmitidos a partir de la señal $s(t)$? ¿Hay ISI?

Dado que la generación de bits es aleatoria, el estudio de las propiedades de esta señal se encuadra en el campo del procesamiento estocástico. La secuencia de símbolos generada a la salida del *mapper* también es un proceso estocástico. El filtro transmisor es un sistema LTI excitado con un proceso estocástico.

Una técnica para analizar este tipo de sistemas es la *densidad espectral de potencia* (PSD). La PSD evalúa la potencia del proceso en función de la frecuencia (contenido espectral).

En particular, los símbolos generados son ocurrencias de un proceso uniforme y blanco:

- **Uniforme:** todos los símbolos tienen igual probabilidad de ocurrencia.
- **Blanco:** la probabilidad de ocurrencia de un símbolo no depende de los símbolos anteriores.

La PSD de un proceso blanco es esencialmente plana (constante en frecuencia). Para un sistema LTI excitado con un proceso estocástico se cumple

$$\Phi_{yy}(e^{j\omega}) = |G(e^{j\omega})|^2 \Phi_{xx}(e^{j\omega})$$

donde:

- $\Phi_{yy}(e^{j\omega})$ es la PSD de la salida,
- $\Phi_{xx}(e^{j\omega})$ es la PSD de la entrada,
- $G(e^{j\omega})$ es la respuesta en frecuencia del filtro.

Si la entrada es un proceso blanco, entonces $\Phi_{xx}(e^{j\omega})$ es constante y por lo tanto

$$\Phi_{yy}(e^{j\omega}) = |H(e^{j\omega})|^2$$

es decir, la PSD de la salida tiene la forma de la respuesta en frecuencia del filtro elevada al cuadrado.

3. Utilice la función `scipy.signal.welch()` (método de Welch) para calcular la PSD a la entrada del filtro. Grafique el resultado en dB.
4. Grafique la PSD de la salida del filtro y superponga la respuesta en frecuencia del filtro (elevada al cuadrado). Comente brevemente el resultado obtenido.

Nota: Para lograr gráficos comparables, asegúrese de normalizar la potencia de modo que el valor en $f = 0$ sea 0 dB en todos los gráficos.

Ejercicio 4. Considere un sistema de comunicaciones digitales en banda base donde se transmite una secuencia de símbolos $\{a_k\}$ a una velocidad de B símbolos por segundo mediante un filtro conformador de pulsos $h(t)$. Sea $s(t)$ la señal a la salida del transmisor.

El canal está compuesto exclusivamente por un generador de ruido blanco gaussiano aditivo (AWGN). El receptor está formado por un filtro apareado $h^*(-t)$ seguido de un *sampler* con frecuencia de muestreo igual al *symbol rate*. Finalmente, la señal se normaliza para que la amplitud de los símbolos recibidos sea correcta y se pasa por un *slicer* adecuado para la modulación utilizada.

1. Dada la SNR deseada a la entrada del *slicer*, determine cómo se calcula la SNR requerida en el canal. Justifique su respuesta.
2. Dada la SNR por bit deseada en el *slicer* (E_b/N_0), determine cómo se calcula la SNR requerida en el canal a partir de E_b/N_0 .

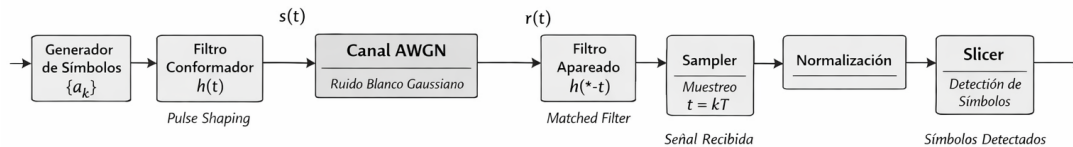


Figure 2: Modelo de envolvente compleja QAM- M .

Escriba en **Python** un simulador que modele el sistema descrito utilizando un filtro *Root Raised Cosine* (RRC) como conformador de pulsos.

Configure un escenario de simulación que permita seleccionar al menos los siguientes parámetros:

- orden de modulación (QPSK y QAM16),
- rolloff del filtro,
- tasa de sobremuestreo,
- valor de E_b/N_0 ,
- cantidad de símbolos transmitidos.

El simulador deberá generar los siguientes resultados:

1. Diagrama de ojo a la salida del transmisor.
2. PSD a la salida del transmisor superpuesta con la PSD a la entrada del filtro apareado.
3. PSD a la entrada del filtro apareado superpuesta con la PSD luego del filtro apareado (antes de decimar la señal).
4. Constelación a la entrada del *slicer*.

5. Histogramas de la parte real e imaginaria de la señal a la entrada del *slicer*.

Utilizando el simulador desarrollado previamente, implemente un script adicional que permita barrer:

- el orden de modulación (QPSK y QAM16),
- el valor de E_b/N_0 .

Simule únicamente algunos valores de E_b/N_0 de modo que se obtengan valores de BER en el rango aproximado

$$10^{-6} \leq \text{BER} \leq 5 \times 10^{-2}.$$

El script deberá generar una figura donde se muestren:

- las curvas teóricas de BER para cada modulación,
- los puntos obtenidos mediante simulación.